

Understanding CSS Counters

CSS counters are variables that can be incremented, decremented, and displayed using CSS rules. They allow you to automatically number elements without hardcoding numbers in HTML. Counters are maintained by CSS alone and enable dynamic, automatic numbering for lists, sections, pages, and more—perfect for creating automatic table of contents, custom numbering systems, and complex outline structures.

Key Point: CSS counters consist of three properties: counter-reset (initialize), counter-increment (increase), and content: counter() (display). Counters are scoped to their reset scope and can be nested, allowing for hierarchical numbering like 1.1, 1.2, 2.1, etc.

CSS Counter Properties Reference

Property	Syntax	Description
counter-reset	counter-reset: name 0;	Initializes a counter to 0 (or specified value)
counter-increment	counter-increment: name;	Increases counter by 1 (or specified amount)
content: counter()	content: counter(name);	Displays counter value (numeric)
content: counters()	content: counters(name, ",");	Displays nested counters with separator

Counter Format Types Reference

Format	Syntax	Output	Example
Decimal (default)	counter(name)	1, 2, 3, 4...	counter(section)
Lowercase letters	counter(name, lower-alpha)	a, b, c, d...	counter(item, lower-alpha)
Uppercase letters	counter(name, upper-alpha)	A, B, C, D...	counter(item, upper-alpha)
Lowercase Roman	counter(name, lower-roman)	i, ii, iii, iv...	counter(chapter, lower-roman)
Uppercase Roman	counter(name, upper-roman)	I, II, III, IV...	counter(chapter, upper-roman)
Custom string	counter(name, "string")	custom1, custom2...	counter(item, "Q")

Practical Examples

Example 1: Basic Counter - Number Sections

```
/* CSS */
.sections { counter-reset: section; }
.sections h3:before { counter-increment: section;
content: "Section " counter(section) ": "; }
```

HTML Usage:

```
<div class="sections">
  <h3>Introduction</h3>
  <h3>Content</h3>
  <h3>Conclusion</h3>
</div>
```

Live Demo:

Section 1: Introduction

Section 2: Content

Section 3: Conclusion

Example 2: Custom List Counter

```
/* CSS */
ol { counter-reset: list-counter; list-style: none; padding-left: 0; }
ol li { counter-increment: list-counter; margin: 10px 0; padding-left: 30px; position: relative; }
ol li:before { content: counter(list-counter) ". "; position: absolute; left: 0; color: #fa709a; font-weight: bold; }
```

HTML Usage:

```
<ol>
  <li>First item</li>
  <li>Second item</li>
  <li>Third item</li>
</ol>
```

Live Demo:

1.

First item in custom numbered list

2.

Second item in custom numbered list

3.

Third item in custom numbered list

Example 3: Alphabetic Counter

```
/* CSS */
ol { counter-reset: alpha-counter; list-style: none; padding-left: 0; }
ol li { counter-increment: alpha-counter; margin: 10px 0; padding-left: 30px; position: relative; }
ol li:before { content: counter(alpha-counter, lower-alpha) " "; position: absolute; left: 0; }
```

HTML Usage:

```
<ol>
  <li>Option A</li>
  <li>Option B</li>
  <li>Option C</li>
</ol>
```

Live Demo:

a)

Option A

b)

Option B

c)

Option C

Example 4: Roman Numeral Counter

```
/* CSS */
.chapters { counter-reset: roman-counter; }
.chapters li { counter-increment: roman-counter; margin: 10px 0; padding-left: 40px; position: relative; list-style: none; }
.chapters li:before { content: counter(roman-counter, upper-roman) ". "; position: absolute; left: 0; font-weight: bold; color: #3498db; }
```

HTML Usage:

```
<ul class="chapters">
  <li>Introduction</li>
  <li>Main Content</li>
  <li>Conclusion</li>
</ul>
```

Live Demo:

I.

Introduction

II.

Main Content

III.

Conclusion

Example 5: Decorated Item Counter

```
/* CSS */
.items { counter-reset: item-counter; list-style: none; padding-left: 0; }
.items li { counter-increment: item-counter; margin: 10px 0; padding-left: 40px; position: relative; }
.items li:before { content: "<div class='item'>Item " counter(item-counter); position: absolute; left: 0; background: #27ae60; color: white; padding: 2px 8px; border-radius: 3px; font-size: 12px; font-weight: bold; }
```

HTML Usage:

```
<ul class="items">
  <li>Completed task</li>
  <li>Completed task</li>
</ul>
```

Live Demo:

✓ Item 1

Completed task

✓ Item 2

Completed task

✓ Item 3

Completed task

Example 6: Step Counter - Numbered Circles

```
/* CSS */
.steps { counter-reset: steps; }
.step { counter-increment: steps; margin: 15px 0; padding-left: 50px; position: relative; }
.step:before { content: counter(steps); position: absolute; left: 0; top: 0; background: #fa709a; color: white; width: 40px; height: 40px; border-radius: 50%; display: flex; align-items: center; justify-content: center; font-weight: bold; font-size: 16px; }
```

HTML Usage:

```
<div class="steps">
  <div class="step">First step description</div>
  <div class="step">Second step description</div>
  <div class="step">Third step description</div>
</div>
```

Live Demo:

1

First step description - Setup your environment

2

Second step description - Configure settings

3

Third step description - Complete installation

Example 7: Nested Counters - Hierarchical Numbering

```
/* CSS */
ul.nested { counter-reset: main-counter; list-style: none; }
ul.nested > li:before { counter-increment: main-counter; content: counter(main-counter) ". "; }
ul.nested ul { counter-reset: sub-counter; list-style: none; }
ul.nested ul li:before { counter-increment: sub-counter; content: counter(main-counter) ". " counter(sub-counter) " "; }
```

HTML Usage:

```
<ul class="nested">
  <li>Main 1
    <ul>
      <li>Sub 1.1</li>
      <li>Sub 1.2</li>
    </ul>
  </li>
</ul>
```

Live Demo:

1.

Main topic 1

1.1

Subtopic 1.1

1.2

Subtopic 1.2

2.

Main topic 2

2.1

Subtopic 2.1

2.2

Subtopic 2.2

Example 8: Page Counter (Document Numbering)

```
/* CSS */
.document { counter-reset: page-num; }
.page { counter-increment: page-num; page-break-after: always; border: 2px solid #ddd; padding: 20px; margin: 10px 0; min-height: 200px; }
.page:after { content: "Page " counter(page-num); display: block; text-align: center; margin-top: 20px; color: #95a5a6; }
```

HTML Usage:

```
<div class="document">
  <div class="page">Page content...</div>
  <div class="page">Page content...</div>
</div>
```

Live Demo:

This is page 1 content

Page 1

This is page 2 content

Page 2

Example 9: counters() Function - Automatic Nesting

```
/* CSS */
ol { counter-reset: item; list-style-type: none; padding-left: 0; }
li { counter-increment: item; margin-bottom: 5px; }
li li:before { content: counters(item, ".") " "; font-weight: bold; }
```

Live Demo:

1.1

Subsection

1.2

Subsection

2

Second section

Example 10: Counter with Text Prefix & Suffix

```
/* CSS */
.instructions { counter-reset: instruction-num; }
.instruction { counter-increment: instruction-num; margin: 15px 0; padding: 15px; background: #f0f0f0; border-radius: 4px; border-left: 4px solid #fa709a; }
.instruction:before { content: "Step " counter(instruction-num) ": "; font-weight: bold; color: #fa709a; }
```

Live Demo:

Step 1:

Read instructions carefully

Step 2:

Follow the process

Step 3:

Complete the task

Example 11: Table Row Counter

```
/* CSS */
table { counter-reset: row-num; width: 100%; border-collapse: collapse; }
tr:first-child:before { counter-increment: row-num; content: counter(row-num) ". "; font-weight: bold; color: #fa709a; }
```

Live Demo:

1.	John Doe	Active
2.	Jane Smith	Inactive
3.	Bob Johnson	Active

Counter Scope & Nesting

Concept	Description	Example
Counter Reset	Initializes counter to 0 (or custom value)	counter-reset: section; (resets to 0)
Counter Increment	Increases counter value	counter-increment: section; (increases by 1)
Nested Counters	Separate counters at different levels	1.1, 1.2, 2.1 structure
Counter Scope	Each reset creates new scope	Nested ul creates sub-counter scope
counters() Function	Combines all levels with separator	counters(item, ",") outputs "1.2.3"

Counter Format Styles

Style	Outputs	Use Case
decimal	1, 2, 3, 4, 5...	Basic numbered lists
lower-alpha	a, b, c, d, e...	Lettered options
upper-alpha	A, B, C, D, E...	Uppercase lettering
lower-roman	i, ii, iii, iv, v...	Lowercase Roman numerals
upper-roman	I, II, III, IV, V...	Chapters and sections

Best Practices

1. Use Meaningful Counter Names:

Name counters descriptively (e.g., chapter-num, section-num) so code is self-documenting.

2. Reset at Appropriate Levels:

Reset counters at the correct scope to avoid numbering conflicts in nested lists.

3. Combine with ::before::after:

Use pseudo-elements to display counters without modifying HTML structure.

4. Consider Accessibility:

Ensure counter values are meaningful in screen readers. Don't rely on counters for semantic meaning.

5. Test Nested Levels:

Thoroughly test nested counters to ensure correct numbering across all levels.

Common Issues & Solutions

Issue 1: Counter Not Incrementing

Problem: Counter value stays at 0

Solution: Ensure counter-increment is in the right selector (usually element or ::before pseudo-element).

Issue 2: Nested Counter Resets Too Early

Problem: Sub-counter doesn't maintain separate values

Solution: Place counter-reset on child element, not parent. Each level needs its own reset point.

Issue 3: counters() Doesn't Display Correctly

Problem: Nested counter structure doesn't appear

Solution: Ensure all parent counters are incremented before display. Use consistent separator (e.g., ".")

Real-World Use Cases

Use Case	Solution
Automatic table of contents	Number headings with counters, display in TOC with content: counter()
Legal document numbering	Use nested counters for sections (1.1.1) and roman numerals
Tutorial steps	Display step numbers in circles or badges using ::before
Bug/issue tracking	Auto-number issues without server-side IDs
FAQ sections	Auto-number questions and answers independently
Page numbers in print	Use page counter with @page rule for print layouts

Summary

CSS counters provide an elegant solution for automatic numbering without modifying HTML or hardcoding values. From simple list numbering to complex hierarchical structures, counters offer flexible, maintainable document layouts. Combined with CSS pseudo-elements and formatting options, counters create powerful automatic numbering systems perfect for tables of contents, legal documents, tutorials, and more.

Key Takeaway: Master CSS counters for automatic numbering systems. Use counter-reset to initialize, counter-increment to track, and content: counter() to display. Leverage nested counters for hierarchical numbering (1.2.3). Use different counter formats (roman numerals, letters) for different document sections. Combine with pseudo-elements for flexible, semantic numbering.

CSS Counters Guide | All examples are responsive and work on any device | Last updated: 2026